

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL-1 EXPERIMENTS

Course Code:	CSA1221	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3Statement:	<i>Design processor organization by constructing pipelined Datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	Jeevan Raaja S	Reg.No.:	192421005
Department:	Information Technology	Date:	09/08/2026

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct:

I Jeevan Raaja S (192421005) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Jeevan Raaja S

MARKS ALLOCATION

	Understanding of Concepts(20%)	Experimental Design(20%)	Use of Tools(25%)	Analysis of Results(20%)	Documentation(15 %)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. Register Transfer Operations and Basic ALU Operations

Based on the **Q1 experiment** in your uploaded assessment, the task is to design and simulate register-transfer operations together with **addition, subtraction, AND, and OR**, using multiple registers and analyzing the role of the ALU.

1. Aim

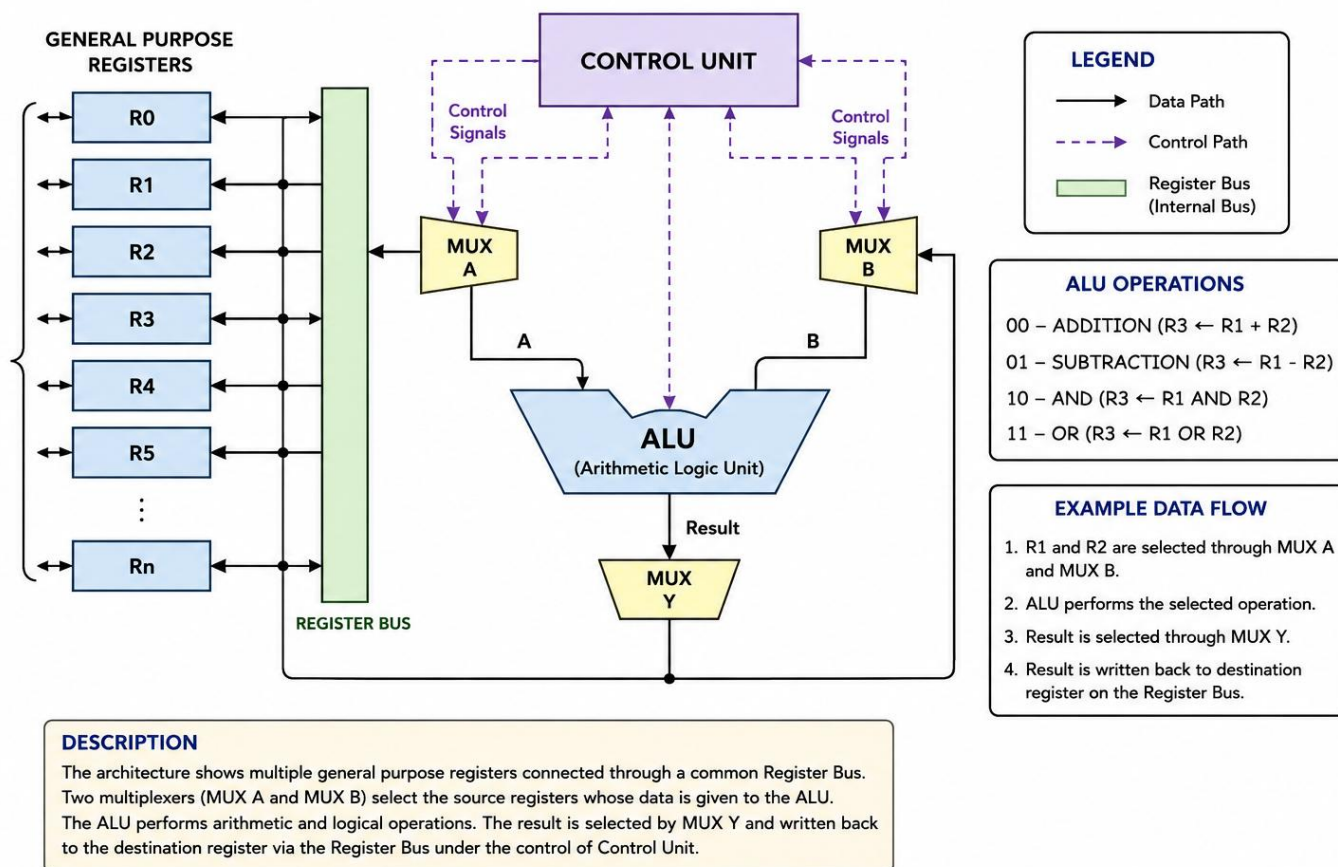
To design and simulate a simple processor datapath that performs **register transfer, arithmetic, and logic operations** using multiple registers and an **Arithmetic Logic Unit (ALU)**.

2. Objectives

- Perform data transfer between registers.
- Perform addition and subtraction using the ALU.
- Perform AND and OR logical operations.
- Understand how registers communicate through the datapath.
- Analyze the sequence of operations during instruction execution.
- Understand the role of the ALU in processing data.

3. Basic Architecture

REGISTER TRANSFER AND ALU ARCHITECTURE



Here, **R1** and **R2** provide operands to the ALU, while **R3** stores the result.

4. Register Transfer Operations

Register transfer means moving data from one register to another.

For example:

$R1 \leftarrow R2$

This means the contents of **R2 are copied into R1**.

Another example:

$R3 \leftarrow R1$

The contents of R1 are transferred to R3.

The original contents of the source register are not changed.

5. ALU Operations

The ALU performs the following operations:

Operation	RTL Representation	Description
Addition	$R3 \leftarrow R1 + R2$	Adds R1 and R2
Subtraction	$R3 \leftarrow R1 - R2$	Subtracts R2 from R1
AND	$R3 \leftarrow R1 \text{ AND } R2$	Performs bitwise AND
OR	$R3 \leftarrow R1 \text{ OR } R2$	Performs bitwise OR

6. Example Execution

Assume:

$R1 = 12$

$R2 = 5$

Addition

$R3 \leftarrow R1 + R2$

$R3 \leftarrow 12 + 5$

$R3 = 17$

Subtraction

$R3 \leftarrow R1 - R2$

$R3 \leftarrow 12 - 5$

$R3 = 7$

AND Operation

12 in binary:

1100

5 in binary:

0101

Therefore:

1100

0101

0100

So:

$R3 = 4$

OR Operation

1100

0101

1101

Therefore:

$R3 = 13$

7. Sample Register-Transfer Sequence

A possible execution sequence is:

$R1 \leftarrow 12$

$R2 \leftarrow 5$

$R3 \leftarrow R1 + R2$

$R4 \leftarrow R1 - R2$

$R5 \leftarrow R1 \text{ AND } R2$

$R6 \leftarrow R1 \text{ OR } R2$

The registers contain:

Register	Operation	Result
R1	Input	12
R2	Input	5
R3	Addition	17
R4	Subtraction	7
R5	AND	4
R6	OR	13

8. Role of the ALU

The **ALU is the main computational unit** of the datapath. It receives operands from registers, performs the selected arithmetic or logical operation, and sends the result to the destination register.

The operation can be represented as:

Source Registers

↓

Operands

↓

ALU

↓

Selected Operation

↓

Result



Destination Register

The control unit determines which ALU operation should be performed and which registers should provide and receive the data.

9. Analysis of Results

The experiment demonstrates that register-transfer operations provide the movement of data within the processor, while the ALU performs the required computation. For example, when R1 contains 12 and R2 contains 5, the ALU produces **17 for addition, 7 for subtraction, 4 for AND, and 13 for OR.**

Using separate registers for operands and results allows the processor to maintain intermediate values and perform a sequence of operations systematically. Thus, the combination of **registers, datapath, control signals, and ALU** forms the basic structure required for processor data processing.

10. Conclusion

The register-transfer and ALU simulation successfully demonstrates how data moves between multiple registers and how arithmetic and logical operations are performed. The experiment shows that the **ALU is responsible for computation**, while registers store operands and intermediate results. This provides a fundamental understanding of processor datapath organization and register-level execution.

Q2. Single-Bus and Multi-Bus Organization

This answer is structured to target Level 4 (Exemplary) of your rubric by including the architecture, systematic data-transfer process, comparison, performance analysis, and conclusion. The experiment specifically requires comparison of single-bus and multi-bus organizations for transfers between registers, memory, and I/O.

1. Aim

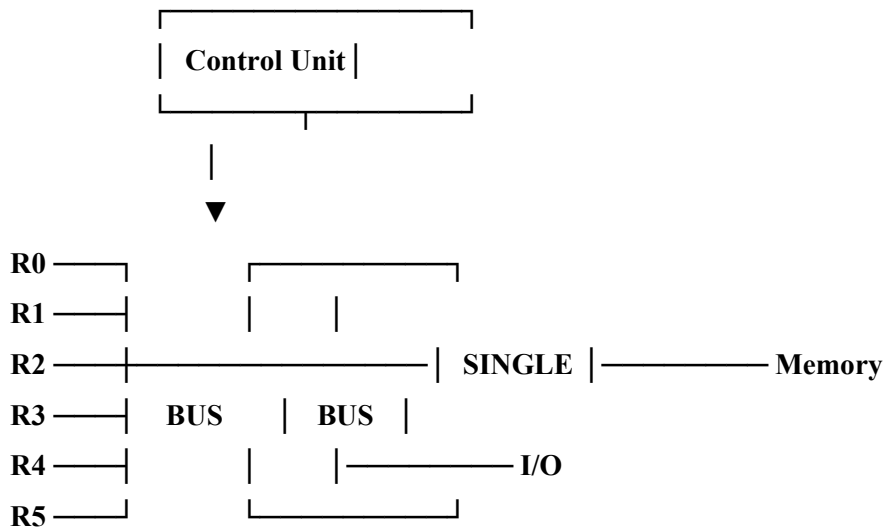
To design and simulate single-bus and multi-bus computer organizations, perform data transfers between registers, memory, and I/O, and compare their execution time, efficiency, parallelism, and bottlenecks.

2. Objectives

- Understand single-bus organization.
- Understand multi-bus organization.
- Perform register-to-register data transfers.
- Perform memory read/write operations.
- Perform I/O data transfers.
- Compare the execution time of both architectures.
- Analyze how multiple buses improve parallel data transfer.

3. Single-Bus Organization

In a single-bus organization, registers, memory and I/O devices communicate through one common bus.



Only one major data transfer can use the shared bus at a time.

For example:

$R1 \rightarrow \text{Bus} \rightarrow R2$

Then another operation must wait for the bus.

4. Example Single-Bus Data Transfer

Consider:

$R1 \leftarrow R2$

$R3 \leftarrow R4$

The transfers have to occur sequentially:

Time T1 : $R2 \rightarrow \text{Bus} \rightarrow R1$

Time T2 : $R4 \rightarrow \text{Bus} \rightarrow R3$

Therefore, two independent transfers require multiple bus cycles.

Memory Transfer

For a memory read:

$\text{MAR} \leftarrow \text{Address}$

$\text{Memory} \rightarrow \text{MDR}$

$\text{MDR} \rightarrow \text{Destination Register}$

Each transfer uses the common bus.

I/O Transfer

For an input operation:

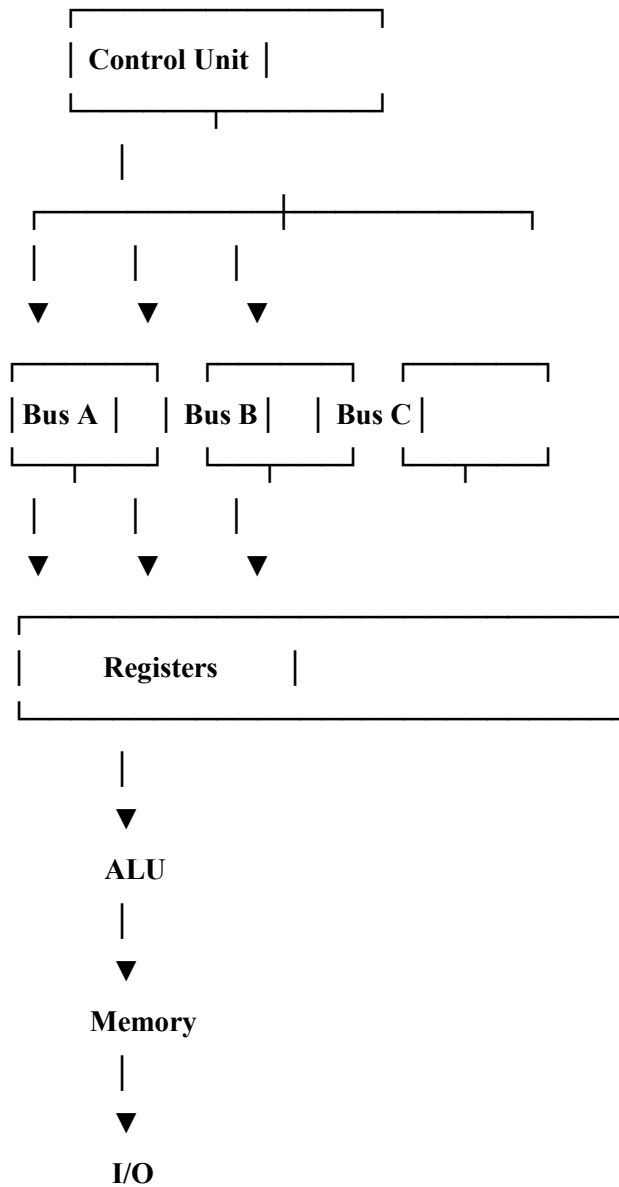
$\text{I/O Device} \rightarrow \text{Bus} \rightarrow \text{Register}$

Again, the shared bus becomes the communication path.

5. Multi-Bus Organization

A multi-bus organization uses two or more buses so that multiple data transfers can take place simultaneously.

A typical three-bus structure is:



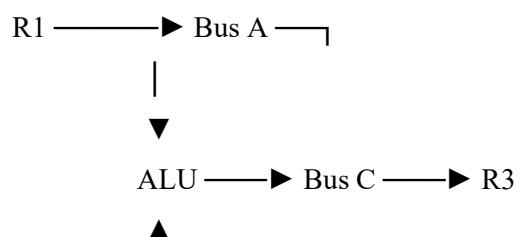
The important advantage is that multiple operands can be transferred at the same time.

6. Example Multi-Bus Transfer

Consider:

$$R3 \leftarrow R1 + R2$$

With two source buses:



The theoretical reduction is:

Thus, in this simplified example, the multi-bus organization can reduce the transfer time by approximately 33.33%.

The exact improvement depends on the number of buses, instruction sequence, memory access time, and control implementation.

9. Why Multi-Bus Improves Performance

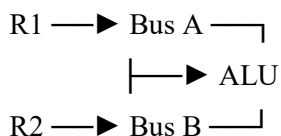
Multiple buses reduce the major limitation of a single-bus architecture: bus contention.

For example, in a single-bus system:



Only one transfer can effectively use the shared path at a particular time.

In a multi-bus system:



Both operands can reach the ALU concurrently.

Therefore, multiple buses:

- Reduce waiting time.
- Increase data-transfer parallelism.
- Reduce bus contention.
- Improve ALU utilization.
- Increase instruction throughput.
- Improve overall processor efficiency.

10. Experimental Analysis

The experiment demonstrates that the single-bus architecture is simpler and cheaper, but its common communication path creates a bottleneck when several components require data transfers.

The multi-bus architecture provides greater parallelism because independent transfers can occur through separate buses. This reduces the number of required transfer cycles and improves processor performance.

However, the improvement comes at the cost of increased hardware, control complexity, and implementation cost.

Thus, the choice depends on the design requirement: single-bus organization favors simplicity and lower cost, whereas multi-bus organization favors higher performance and parallel data movement.

11. Conclusion

The single-bus and multi-bus organizations were analyzed for register, memory, and I/O data transfers. The single-bus structure provides a simple and economical architecture but suffers from bus contention and sequential transfers. The multi-bus structure allows parallel data movement, reducing transfer time and

improving resource utilization.

Therefore, multi-bus organization provides better performance and scalability, while single-bus organization is preferable when simplicity and lower hardware complexity are the primary requirements.

Q3. Simulation of a 5-Stage Instruction Pipeline

This answer is based on **Q3 of your uploaded assessment**, which asks you to simulate a **5-stage instruction pipeline**, measure **speedup and throughput**, and compare it with a **non-pipelined system**.

1. Aim

To design and simulate a **5-stage instruction pipeline** consisting of **Fetch, Decode, Execute, Memory, and Write-back**, and evaluate its performance by comparing pipelined and non-pipelined execution.

2. Objectives

- Understand the five stages of instruction execution.
- Simulate multiple instructions passing through the pipeline.
- Compare pipelined and non-pipelined execution.
- Calculate **speedup**.
- Calculate **instruction throughput**.
- Analyze the effectiveness of pipelining.

3. Five Pipeline Stages

The processor instruction is divided into five stages:

Stage	Name	Function
IF	Instruction Fetch	Fetches instruction from memory
ID	Instruction Decode	Decodes instruction and reads operands
EX	Execute	Performs ALU operation
MEM	Memory	Performs memory read/write if required
WB	Write-back	Writes result into destination register

The overall flow is:

Instruction

↓

IF

↓

ID

↓

EX

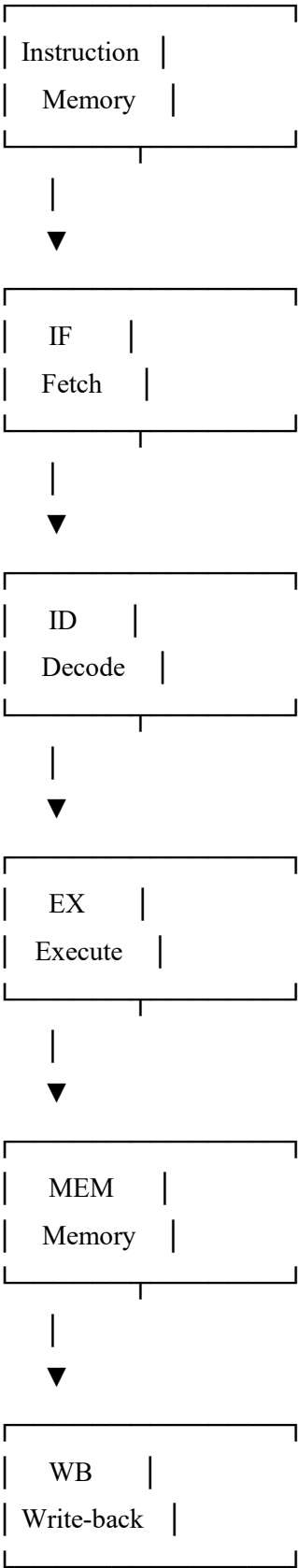
↓

MEM

↓

WB

4. Pipeline Architecture



5. Instructions Used for Simulation

Consider five instructions:

I1: ADD R1, R2, R3

I2: SUB R4, R5, R6

I3: AND R7, R8, R9

I4: OR R10, R11, R12

I5: ADD R13, R14, R15

Each instruction passes through all five pipeline stages.

6. Pipeline Execution

The execution can be represented using a timing diagram.

Cycle → 1 2 3 4 5 6 7 8 9

I1 IF ID EX MEM WB

I2 IF ID EX MEM WB

I3 IF ID EX MEM WB

I4 IF ID EX MEM WB

I5 IF ID EX MEM WB

The important point is that **multiple instructions are processed simultaneously in different stages**.

For example, during Cycle 3:

I1 → EX

I2 → ID

I3 → IF

Therefore, the processor does not wait for I1 to completely finish before starting I2 and I3.

7. Non-Pipelined Execution

In a non-pipelined processor, an instruction completes all five stages before the next instruction begins.

For five instructions:

I1 → IF → ID → EX → MEM → WB

I2 → IF → ID → EX → MEM → WB

I3 → IF → ID → EX → MEM → WB

I4 → IF → ID → EX → MEM → WB

I5 → IF → ID → EX → MEM → WB

If each stage requires one clock cycle:

8. Pipelined Execution

For a pipeline with **5 stages**, the first instruction requires five cycles to complete.

After that, one instruction can complete every cycle.

Therefore:

where:

- stages

- instructions

So:

Architecture	Cycles
Non-pipelined	25
Pipelined	9

9. Speedup

Speedup is calculated as:

Therefore:

Thus, for this idealized five-instruction example, the pipeline provides approximately **2.78× speedup**.

10. Throughput

Throughput represents the number of instructions completed per unit of time.

Assuming one clock cycle is the time unit:

Non-pipelined

Pipelined

Therefore, pipelining significantly improves instruction throughput.

11. Performance Comparison

Parameter	Non-Pipelined	Pipelined
Pipeline stages	5	5
Instructions	5	5
Total cycles	25	9
Throughput	0.20 instr/cycle	0.556 instr/cycle
Speedup	1×	2.78×
Parallel execution	No	Yes
Hardware complexity	Lower	Higher

12. Analysis of Results

The simulation demonstrates the major advantage of instruction pipelining. In a non-pipelined processor, each instruction occupies the processor for all five stages before the next instruction begins.

In the pipelined processor, different instructions can occupy different stages simultaneously. For example, while one instruction is executing in the **EX stage**, another can be decoded in **ID** and another can be fetched in **IF**.

For five instructions, the ideal execution time decreases from **25 cycles to 9 cycles**, producing a calculated speedup of approximately **2.78×**. The throughput also increases from **0.20 to approximately 0.556 instructions per cycle**.

The improvement is greatest when the pipeline stages are balanced and there are few interruptions such as hazards or stalls.

13. Conclusion

The 5-stage instruction pipeline was successfully analyzed using the stages **IF, ID, EX, MEM, and WB**. Compared with non-pipelined execution, pipelining allows multiple instructions to be processed concurrently at different stages.

For the five-instruction example, execution is reduced from **25 cycles to 9 cycles**, giving an idealized speedup of approximately **2.78×**. Therefore, pipelining improves **instruction throughput and processor performance**, although practical performance can be reduced by hazards, stalls, and other pipeline limitations.

Q4. Pipeline Hazards and Hazard Resolution Techniques

This answer is based on **Q4 of your uploaded Computer Architecture assessment**, which requires a pipeline scenario containing **data, control, and structural hazards**, followed by techniques such as **stalling, data forwarding, and branch prediction** to improve performance.

1. Aim

To develop and analyze a pipelined processor scenario in which **data hazards, control hazards, and structural hazards** occur, and implement appropriate techniques to resolve these hazards and improve pipeline performance.

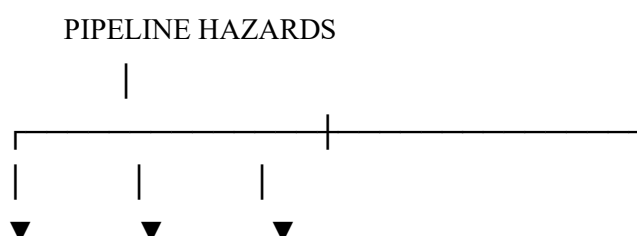
2. Objectives

- Identify different types of pipeline hazards.
- Analyze their effect on instruction execution.
- Demonstrate data hazards using dependent instructions.
- Demonstrate control hazards using branch instructions.
- Demonstrate structural hazards using shared hardware resources.
- Apply stalling, data forwarding, and branch prediction.
- Compare pipeline performance before and after hazard resolution.

3. Pipeline Hazards

A **pipeline hazard** is a condition that prevents the next instruction from executing in its intended pipeline cycle.

The three major types are:



Data Hazard Control Hazard Structural Hazard

Types

Hazard	Main Cause	Solution
Data	Dependency between instructions	Forwarding / Stalling
Control	Branch or jump instruction	Branch Prediction
Structural	Hardware resource conflict	Additional resources / Stalling

4. Data Hazard

A data hazard occurs when one instruction depends on the result produced by a previous instruction.

Consider:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5

Here, **I2 requires R1**, but R1 is produced by I1.

Pipeline without Forwarding

Cycle → 1 2 3 4 5 6

I1 IF ID EX MEM WB

I2 IF ID ST EX MEM WB

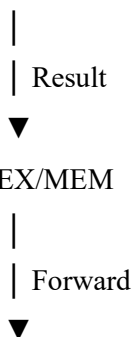
ST represents a stall.

The second instruction must wait until the required data becomes available.

5. Data Forwarding

Instead of waiting for the result to be written into the register file, the result can be directly forwarded from one pipeline stage to another.

I1: ADD R1, R2, R3



I2: SUB R4, R1, R5

The forwarded value is directly supplied to the ALU for I2.

With Forwarding

Cycle → 1 2 3 4 5

I1 IF ID EX MEM WB

I2 IF ID EX MEM WB

Forwarding reduces unnecessary stalls and improves pipeline efficiency.

6. Control Hazard

A control hazard occurs when the processor encounters a **branch or jump instruction**.

Example:

I1: ADD R1, R2, R3

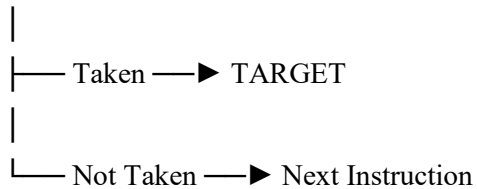
I2: BEQ R1, R0, TARGET

I3: SUB R4, R5, R6

The processor may not immediately know whether the branch will be taken.

Therefore, the processor may fetch the wrong instruction.

Branch



If the prediction is wrong, incorrectly fetched instructions must be removed from the pipeline.

7. Branch Prediction

Branch prediction attempts to predict whether a branch will be taken before the branch condition is completely evaluated.

Example:

BEQ R1, R0, TARGET

Prediction:

Branch Taken

↓

Fetch TARGET instruction

If the prediction is correct, the pipeline continues without a major interruption.

If prediction is incorrect:

Wrong instructions

↓

Flush

↓

Correct instruction

Thus, branch prediction reduces the performance penalty caused by control hazards.

8. Combined Hazard Scenario

Consider the following instructions:

I1: LW R1, 0(R2)

I2: ADD R3, R1, R4

I3: BEQ R3, R0, TARGET

I4: SUB R5, R6, R7

This sequence can produce multiple hazards.

Data Hazard

I1 → produces R1

I2 → requires R1

Control Hazard

I3 → branch instruction

The processor does not immediately know whether it should execute I4 or jump to TARGET.

Structural Hazard

If instruction fetch and memory access share one memory resource, a conflict may occur when:

IF stage → accesses memory

MEM stage → accesses memory

9. Performance Comparison

Consider an ideal pipeline that requires **9 cycles** for five instructions, as in Q3.

If hazards introduce three additional stall/penalty cycles:

Therefore:

Without hazard penalties

With hazards

The additional:

represent the performance penalty caused by hazards.

With forwarding, branch prediction, and appropriate resource design, these additional cycles can be reduced.

10. Conclusion

The experiment demonstrates that **data, control, and structural hazards** can interrupt normal pipeline execution and reduce processor performance. **Data forwarding** reduces data-dependent stalls, **branch prediction** minimizes control-hazard penalties, and **additional hardware resources** help resolve structural conflicts.

Thus, combining these techniques allows the processor to maintain a higher level of parallel instruction execution and achieve performance closer to the ideal pipeline.

Q5. Superscalar Execution, Out-of-Order Execution, Speculative Execution and SMT

This answer is based on **Q5 of your uploaded Computer Architecture assessment**, which asks you to analyze superscalar execution, out-of-order and speculative execution, and extend the study to **hyper-threading and simultaneous multithreading (SMT)**. It also requires comparison of instruction throughput and resource utilization.

1. Aim

To analyze a processor supporting **superscalar execution**, **out-of-order execution**, and **speculative execution**, and study how **hyper-threading and simultaneous multithreading (SMT)** improve instruction throughput and processor resource utilization.

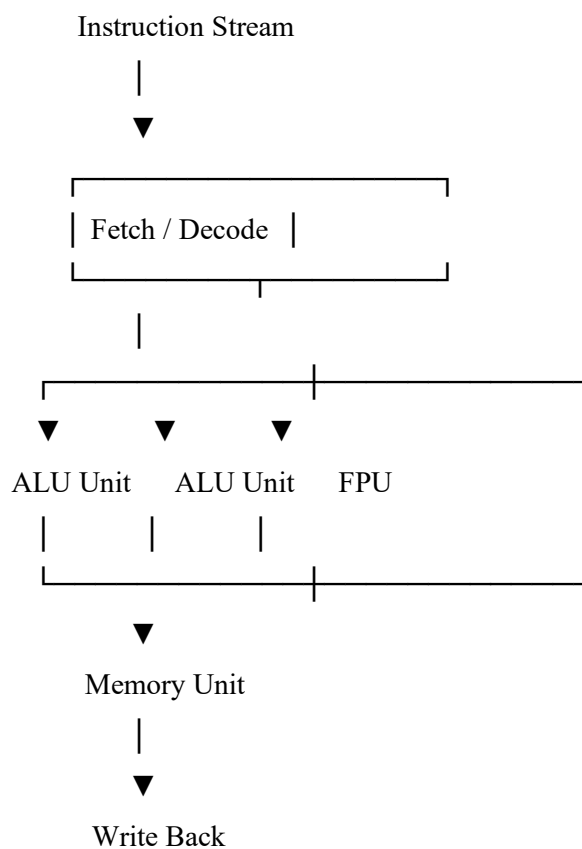
2. Objectives

- Understand superscalar processor organization.
- Analyze parallel execution of multiple instructions.
- Understand out-of-order execution.
- Study speculative execution.
- Understand hyper-threading and SMT.
- Compare instruction throughput and resource utilization.
- Analyze the advantages and limitations of these techniques.

3. Superscalar Architecture

A **superscalar processor** can issue and execute more than one instruction during a single clock cycle.

For example, a processor with a **4-issue superscalar design** can potentially issue up to four independent instructions per cycle.



The processor identifies independent instructions and executes them using available functional units.

4. Example of Superscalar Execution

Consider:

I1: ADD R1, R2, R3

I2: SUB R4, R5, R6

I3: AND R7, R8, R9

I4: OR R10, R11, R12

These instructions are independent.

A superscalar processor may execute them as:

Cycle 1:

I1 → ALU 1

I2 → ALU 2

I3 → ALU 3

I4 → ALU 4

Instead of executing one instruction per cycle, multiple instructions can be executed simultaneously.

This increases **instruction-level parallelism (ILP)**.

5. Out-of-Order Execution

In **out-of-order execution**, instructions are executed when their required operands and resources are available rather than strictly following their original program order.

Consider:

I1: LOAD R1, Memory

I2: ADD R4, R5, R6

I3: SUB R7, R1, R8

I3 depends on I1 because it requires R1.

However, I2 is independent.

Therefore, the processor can execute:

I1 → Waiting for memory

I2 → Execute immediately

I3 → Wait for R1

This avoids keeping the execution units idle.

Basic flow

Instructions



Decode



Instruction Queue



Check Dependencies



Available Instructions



Execute Out-of-Order

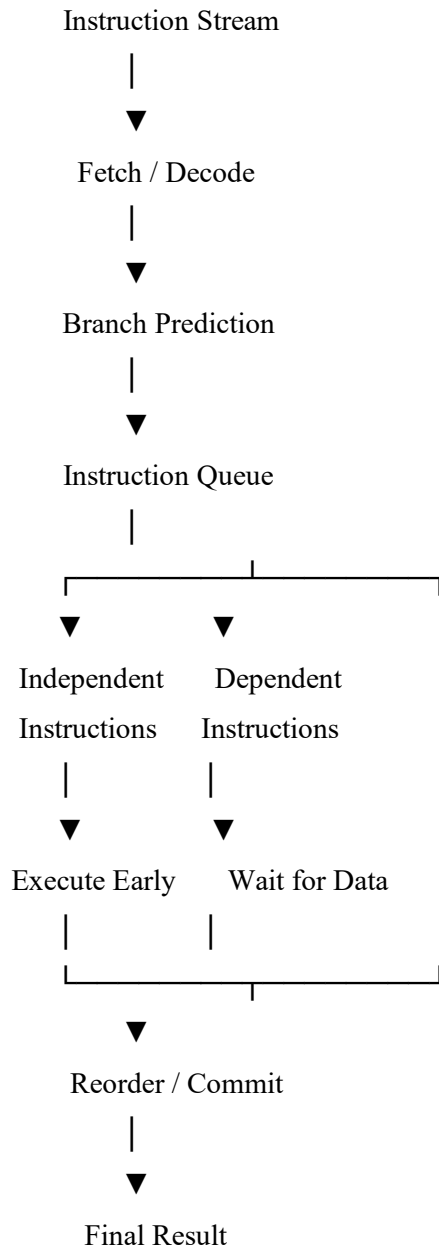


Reorder / Commit

The processor still maintains the correct final program result even though instructions may execute in a different order.

6. Superscalar + Out-of-Order + Speculative Execution

These techniques can work together.



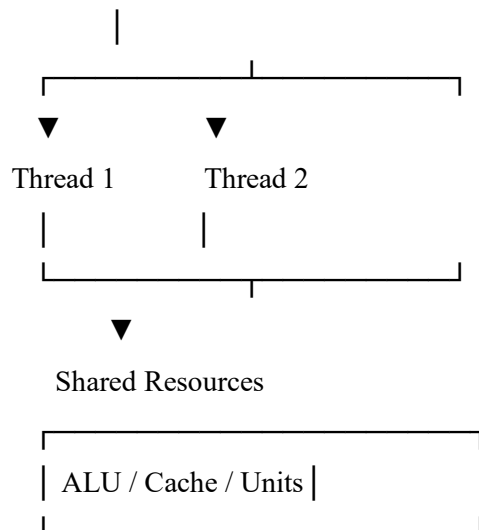
This combination allows the processor to exploit a high degree of parallelism.

7. Hyper-Threading

Hyper-threading is a technology that allows a single physical processor core to appear as multiple logical processors to the operating system.

The logical threads share many physical resources of the core.

Physical CPU Core

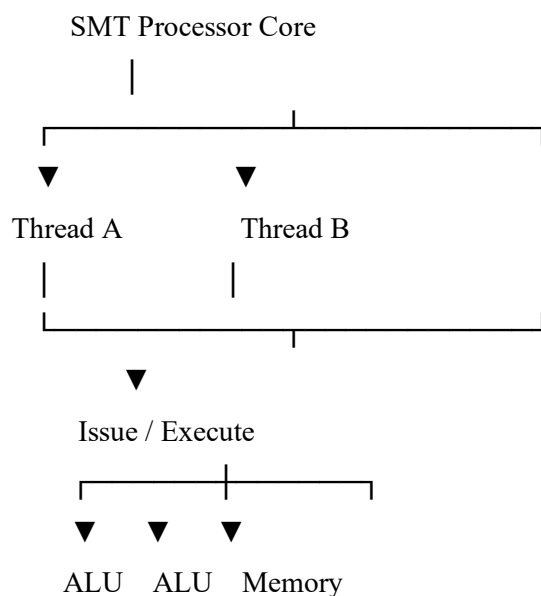


If Thread 1 is waiting for memory, the processor can use available resources to execute Thread 2. This can improve resource utilization.

8. Simultaneous Multithreading (SMT)

SMT allows instructions from multiple threads to be issued and executed simultaneously on the same processor core when sufficient resources are available.

For example:



Instead of allowing only one thread to use the execution resources at a time, SMT can select instructions from multiple threads.

9. Conclusion

The experiment demonstrates that modern processors use several techniques to exploit parallelism.

Superscalar execution allows multiple instructions to execute in parallel, while **out-of-order execution** improves efficiency by executing ready instructions without waiting for earlier stalled instructions.

Speculative execution reduces branch-related delays, and **hyper-threading/SMT** allows multiple threads to share processor resources. Together, these techniques can significantly improve instruction throughput and

resource utilization compared with simple sequential execution.

However, the actual performance improvement depends on factors such as **instruction dependencies**, **branch prediction accuracy**, **memory latency**, **cache behavior**, and **available execution resources**.